airos2.0 改错案例集 —— 设备接入与通信中间件

> 从 193 机 `/work/sunboxue/airos2.0` 的 **1040 个提交**里，只挑**你简历上那三条**对应的模块：
>
> - 组 1 第 1 条「设备抽象与插件化接入」→ `base/device_connect/` 、 `base/plugin/` 、 `middleware/device_service/`
> - 组 1 第 2 条「多路视频流接入」→ `middleware/device_service/modules/ipcamera/` 、 `om_component` 里的相机事件
> - 组 1 第 3 条「设备与平台运维」→ `middleware/protocol/om/` 、 `middleware/protocol/ccindex/` （MEC 自检、MQTT 连接、OTA）
>
> 每个案例都是**真实提交 + 真实 diff**，标了 commit 可以回查。 原始材料在 `fixes/` 目录，全量 diff 在 `_全量patch.patch` 。

怎么用这份文档

- **按你简历的顺序看**：第一~三部分正好对应你三条，第四~五部分是它们底下共用的并发与协议基础。
- **面试怎么用**：每个案例末尾有「能学到什么」和「面试可以这么说」。 ⚠️ 这些**都是同事的活儿，不是你做的** —— 你的作用是"我读过这块的提交历史，知道它踩过哪些坑"，被追问细节时能说上来，但**不能说"我修的"**。
- **共 20 条**，全部来自 `device_service` / `protocol` / `ipcamera` 三个模块的 fix 类提交。

全景：这几个模块的错集中在哪

类型	数量	案例	特征	危险度
配置与字段	6	1-2 1-3 1-4 2-2 5-2 2-3	改了字段/配置，配套的地方没跟上，编译能过、值悄悄丢	★★★★
连接与心跳	4	3-1 ~ 3-4	现场才现形：显示连着实际断了、反复被踢、数据整批缺	★★★★★
进程退出与启动	3	4-3 4-4 4-5	平时没事，重启/下线时段错误或卡死	★★★★★
并发与内存归属	3	4-1 4-2 4-6	偶发、压测才出、core dump 看不出原因	★★★★★
遍历与重复声明	2	2-1 1-1	只处理第一条 / 同一变量声明两遍	★★★★
单位与标签	2	5-1 5-3	数值对但单位或档位标错，对端全错	★★★

一句话概括：**这个项目的错，一大半是"改了一处，忘了配套的另外三处"** —— 结构体加字段要改序列化/反序列化/打印，配置加一项要在两个模块同步，改了返回值语义要检查所有分支。

第一部分：设备抽象与插件化接入

对应简历：「抽象出统一的设备接口…把不同厂商的设备接入收敛到一套插件化框架」。 这一部分的错，几乎全是**"结构体/配置改了，配套的地方没跟上"**。

1-1 同一行变量声明写了两遍 —— 插件编译报"命名重复"

commit：`267fb52`（2026-05-09，zhangenwei）· `middleware/protocol/spill_reporter/spill_reporter.cc`

现象

编译 `spill_reporter`（道路遗撒上报插件）时报命名重复，编译不过。

原来的写法

```
{
    //上传图片路径：crossNo + 日期yyyymmdd + 事件ID + 相机ID + jpg文件
    std::string remote_pictrue_path = dateStr;
    std::string remote_pictrue_name = m_crossNo + "_" + temp_devno + "_" + scpe.ID + ".jpg";

    std::vector<std::string> picture_paths = split_picture_paths(event_info.event_picture());
    std::string remote_pictrue_name = m_crossNo + "_" + temp_devno + "_" + scpe.ID + ".jpg";   // ← 又声明一次
```

为什么错

同一个作用域里 `remote_pictrue_name` 声明了两次。**这不是"写错"，是合并/复制粘贴的残留** —— 大概率是两个人分别给这段加了同一行，git 合并时都保留了。

值得注意：**中间隔了别的语句，所以肉眼扫一遍不容易发现。**两次声明之间还有 `remote_pictrue_path`（没被用到）和 `picture_paths`。

改法

把第一处注释掉（没删，说明作者也不确定哪一处是"新的"）：

```
-    std::string remote_pictrue_name = m_crossNo + "_" + temp_devno + "_" + scpe.ID + ".jpg";
+    //std::string remote_pictrue_name = m_crossNo + "_" + temp_devno + "_" + scpe.ID + ".jpg";
```

能学到什么

- **编译错误是最好的一类错误** —— 它替你发现了合并残留。真正危险的是这类重复**不报错**的情况 （比如两次 `x = 1;` 只是覆盖，编译器不吭声）。
- **合并冲突不要靠"看起来对"放过**。同一段出现两个高度相似的语句，八成是两边各加了一份。
- 这段还留了个 `remote_pictrue_path`（拼写也错了，picture）声明后没用 —— 这类死变量在插件代码里很常见，它们是"改到一半"的化石。

> **面试可以这么说**：插件层代码大多是各人各加一段，合并残留是最常见的编译错误来源。我印象比较深的是 有一处同一个变量被声明了两遍，中间还夹着别的语句，肉眼很容易漏。这类问题编译器帮忙兜住了， 但如果是两处赋值，编译器不会报错，只会在运行时表现为"值莫名其妙被覆盖"，那就难查了。

1-2 结构体加一个字段，要改四个地方

commit：`6daadbe`（2026-04-03，zhangenwei）· `middleware/protocol/om/configer/configer_om_mec.h`

背景

MEC 自检的每个检测项都有一个 `TimeRule`（超时规则）结构体。这次要给每一项加一个"最大超时容忍"字段 `maxTimeoutTolerance`（判断是不是真的连续超时，避免单次抖动就告警）。

一个字段要改四处

```
struct TimeRule : public afl::base::SerializableData
{
    uint64_t timeoutSecond;          // 超时触发阈值（秒）
    uint64_t recoverySecond;         // 恢复所需持续时间（秒）
    bool     enable;                 // 是否检测
    uint64_t maxTimeoutTolerance;    // ① 加成员
private:
    virtual void serialize(json &j) override
    {
        JsonSerialize(timeoutSecond,     "timeoutSecond",     j, false);
        JsonSerialize(recoverySecond,    "recoverySecond",    j, false);
        JsonSerialize(enable,            "enable",            j, false);
        JsonSerialize(maxTimeoutTolerance, "maxTimeoutTolerance", j, false);   // ② 序列化
    }
    virtual void deserialize(const json &j) override
    {
        JsonDeserialize(timeoutSecond,     "timeoutSecond",     j, noUse_isEmptyFlag);
        JsonDeserialize(recoverySecond,    "recoverySecond",    j, noUse_isEmptyFlag);
        JsonDeserialize(enable,            "enable",            j, noUse_isEmptyFlag);
        JsonDeserialize(maxTimeoutTolerance, "maxTimeoutTolerance", j, noUse_isEmptyFlag);  // ③ 反序列化
    }
public:
    std::string to_string() const {
        ss << "enable: " << (enable?"true":"false") << std::endl;
        ss << "maxTimeoutTolerance: " << maxTimeoutTolerance << std::endl;      // ④ 打印
    }
};
```

少改任何一处，症状都不一样：

漏了哪处	症状
① 成员	编译不过（还算幸运）
② 序列化	配置存进文件时丢这个字段，重启后恢复默认值
③ 反序列化	配置文件里写了也不生效，永远用代码里的默认值
④ 打印	日志里看不到它的实际值，排障时瞎猜

能学到什么

- **这类"数据类"结构体，字段是"四处一套"的**：成员 / 序列化 / 反序列化 / 打印。 改的时候要养成"数一数"的习惯，而不是跟着编译器走 —— 因为**漏了 ②③④ 都能编过。**
- 更彻底的做法是用宏或者代码生成，让一处声明自动展开成四处。这个项目用的 `JsonSerialize` 宏 已经省了一半事（至少序列化/反序列化成对），但还是得手写两遍。
- 排障启示：**如果"配置改了不生效"，先看反序列化那一行有没有加。**这是最高频的原因。

> **面试可以这么说**：我读代码时注意到，这类可序列化的配置结构体，加一个字段要同步改四个地方—— 成员、序列化、反序列化、日志打印。只改成员能编过，但配置存不住、或者存了读不回来。 所以遇到"配置改了不生效"，第一反应应该去看反序列化那段有没有漏。

1-3 加了字段忘了给默认值 —— 整项检查永远不启用

commit：`0fc1d6b`（2026-03-11，zhangenwei）· `configer_om_mec.h`

现象

MEC 自检里"点云数据"这一项永远不告警，也不出现在上报结果里。

原来的写法

`MecSelfCheckTime` 的构造函数里，把每一项的阈值都初始化一遍：

```
// 点云数据
cloud_point_data_tr.timeoutSecond = 6;
cloud_point_data_tr.recoverySecond = 3600;
// cloud_point_data_tr.timeoutSecond = 6;
// cloud_point_data_tr.recoverySecond = 6;
                                        // ← 这里空着，enable 没赋值
sensor_cloud_link_tr.timeoutSecond = 5;
sensor_cloud_link_tr.recoverySecond = 5;
```

为什么错

`enable` 是个 `bool` 成员，**在构造函数里没被赋值 = 保持成员声明处的默认值。** 如果声明是 `bool enable;`（没写 `= true`），那它就是个不确定值 / `false`。

而所有自检逻辑的第一句都是：

```
if (mecDeviceData->has_md_radar_cp() && getConfiger().configerMec
        .configerMecSelfCheckTime.cloud_point_data_tr.enable)
{
    // 只有 enable 为 true 才进来
}
```

`enable == false` → **这一项永远不进检测逻辑**，不告警、不上报，安静地消失。

改法

```
cloud_point_data_tr.recoverySecond = 3600;
+cloud_point_data_tr.enable = true;
```

能学到什么

- **构造函数里"逐项赋值"的写法，漏一项不会报错。**这种结构最容易被后续维护搞坏 —— 别人加了一项，复制粘贴了 `timeoutSecond` / `recoverySecond` 两行，忘了 `enable`。
- 更稳的写法是**用默认成员初始化**（`bool enable = true;`），这样"忘了赋值"等于"用默认值"， 而不是"用不确定值"。这个项目里 `TimeRule` 的成员就是裸声明，全靠构造函数兜。
- 排障顺序：**"某个自检项没数据"** → 先查 `enable`，再查阈值，最后才查上游有没有发。 这个顺序能省很多时间。

> **面试可以这么说**：自检框架里每一项都有个 enable 开关，全靠在构造函数里逐项赋值打开。 有一次加新项时漏了赋 enable，结果这一项永远不检测也不上报，日志里什么都没有。 后来我总结，这种"逐项初始化"的结构，默认值应该写在成员声明上，不要依赖构造函数—— 漏一个不会报错，只会静默失效。

1-4 批量赋值时复制粘贴，忘了改变量名

commit：`6daadbe`（2026-04-03，zhangenwei）· `configer_om_mec.h`

现象

给二十多个自检项统一加 `maxTimeoutTolerance = 3`，用的是"复制上一行、改变量名"的手工套路。

原来的写法

```
//实时轨迹数据
ccindex_tm_data_trajectories_tr.timeoutSecond = 3;
ccindex_tm_data_trajectories_tr.recoverySecond = 3;
ccindex_tm_data_trajectories_tr.enable = true;
+sensor_objs_data_tr.maxTimeoutTolerance = 3;              // ← 应该是 ccindex_tm_data_trajectories_tr
```

为什么错

复制粘贴上一行改的，**忘了改变量名。**后果：

- `ccindex_tm_data_trajectories_tr.maxTimeoutTolerance` 保持未初始化（0）
- `sensor_objs_data_tr.maxTimeoutTolerance` 被**重复赋值**（前面已经赋过一次 3，这里再赋 3，看不出问题）

所以**编译不报错、运行不报错、日志看不出**，只有"轨迹数据这一项的超时容忍判断"行为不对。

怎么发现的

大概率是后来查"轨迹数据误告警"时顺着配置一项项对，才发现这一行指向了别的对象。

能学到什么

- **批量同构赋值是复制粘贴 bug 的重灾区**，而且因为值都一样（`= 3`）， 连"数据看着不对"这种线索都没有。**唯一的发现手段是逐行核对变量名。**
- 一百多行这样的赋值，人工核对很难。所以这类代码**天生就该用循环**：

  ```
  for (auto* tr : {&spat_src_data_tr, &sensor_objs_data_tr, &ccindex_tc_data_tr, ...}) {
      tr->maxTimeoutTolerance = 3;
  }
  ```

  用循环就没有"改名字"这个动作，也就不会有这个错。
- 这条和 1-2 是同一个病根的两面：**结构体字段多了之后，人肉维护一定会漏。**

> **面试可以这么说**：我读过一个提交，给二十多个检测项统一加一个阈值字段，是复制粘贴改的， 其中有一行忘了改变量名，指向了另一个对象。因为赋的值都一样，编译和运行都看不出来， 只能逐行核对。这件事让我觉得，这种同构的批量赋值就该写成数组遍历，不要手工复制—— 手工复制一定会漏，而且漏了没人知道。

第二部分：多路视频流接入

对应简历：「支持多路 RTSP 视频流接入...按设备维度管理流」。 这一部分的错集中在**遍历、配置归属、循环变量**。

2-1 `for` 循环推进写错对象 —— 大华相机违法图片只收到 1 张

commit：`eb975a2`（2025-12-03，hezhiyong）· `om_component.cc`

现象

对接大华相机，一起违法事件带多张抓拍图。智路 OS 侧**永远只收到 1 张**，其余全丢。

原来的写法

```
if (!doc.child("EventNotificationAlert").child("ANPR").child("pictureInfoList").empty() && have_jpg)
{
    pugi::xml_node pictureinfolist_node =
        doc.child("EventNotificationAlert").child("ANPR").child("pictureInfoList");

    for (pugi::xml_node temp_node = pictureinfolist_node.first_child();
         temp_node != NULL;
         temp_node = pictureinfolist_node.next_sibling())     // ← 推进的是父节点的兄弟
    {
        if (temp_node.child("fileName").empty() || temp_node.child("absTime").empty()) { ... }
    }
}
```

为什么错

`for` 的第三段（推进语句）写的是 `pictureinfolist_node.next_sibling()`， **这是 `pictureInfoList` 这个父节点的下一个兄弟，不是当前遍历到的 `temp_node` 的兄弟。**

结果：

- 第 1 次迭代：`temp_node = first_child()` → 第 1 张图，正常处理
- 第 2 次迭代：`temp_node = pictureinfolist_node.next_sibling()` → 父节点的下一个兄弟。 在 XML 里 `<pictureInfoList>` 后面通常没有别的兄弟节点 → 返回空 → `temp_node == NULL` → **循环结束**

所以**无论有几张图，只处理第一张。**

改法

一个字母的差别 —— 把 `pictureinfolist_node` 换成 `temp_node`：

```
-    for (pugi::xml_node temp_node = pictureinfolist_node.first_child(); temp_node != NULL; temp_node =
pictureinfolist_node.next_sibling())
+    for (pugi::xml_node temp_node = pictureinfolist_node.first_child(); temp_node != NULL; temp_node =
temp_node.next_sibling())
```

能学到什么

- **`for` 循环三句话，最常见的 bug 在第三句。**初始化写错会直接崩，条件写错会死循环或零次，只有推进写错是"少跑几次"—— 不崩、不报错、不卡死，最难发现。
- **写 `for` 时把三个变量名念一遍**：`init ∈ 条件 ∈ 推进` 必须是同一套对象。这里 `init` 用的是 `pictureinfolist_node.first_child()`，推进却用了 `pictureinfolist_node.next_sibling()`，两个不同的对象。
- 这类 bug 在"只能处理第一条"的现象里很典型。凡是遇到"只处理了第一个/最后一个"，第一反应就该去看循环的推进语句。

> **面试可以这么说**：我们接过一个大华相机的违法事件，一台车带好几张抓拍图，但系统永远只收到一张。根因是 XML 遍历的 for 循环里，推进语句写成了父节点的 next_sibling，而不是当前节点的 next_sibling，所以第二次迭代就退出了。它不崩不报错，只处理一条。凡是"只处理第一条"的现象，我都先看循环的第三句。

2-2 相机内外参的 MQTT topic 挂错了模块

commit：`4767f67`（2026-03-05，zhangenwei）· 8 个文件

现象

运维平台下发"查相机内外参"，设备收不到；设备上报内外参，平台也收不到。topic 名字看着都对。

为什么错

相机内外参这一组 topic，**定义在相机模块的配置里**（`configerCamera.configerTopic`），但**实际收发是在运维（OM）组件里做的。**

两边各自拼前缀：

```
// 原来：用相机模块的前缀 + 相机模块的 topic 名
topicConfiger.Topic_InternalExternalParams_Up =
    topicCameraProfix + getConfiger().configerCamera.configerTopic.Topic_InternalExternalParams_Up;
topicConfiger.Topic_InternalExternalParams_Up_Ack =
    topicCameraProfix + getConfiger().configerCamera.configerTopic.Topic_InternalExternalParams_Up_Ack;
// ... Query / Query_Down 同理
```

问题在于：**`device_service` 里的相机模块和 `protocol` 里的运维模块，用的是两套配置、两个前缀。**运维平台按协议 V3.4 文档的 topic 命名来发布，设备却按相机模块自己的前缀去订阅 → 对不上。

改法

把这一组 topic 的定义**移到运维模块**（`configerMec.configerTopic`），并用运维模块的前缀：

```
-// 相机模块那套（整段注释掉）
-topicConfiger.Topic_InternalExternalParams_Up = topicCameraProfix + ...configerCamera...
+// 运维模块这套
+m_MqttClientConfig.configerMec.configerTopic.Topic_Camera_InterExter_Param_Query_Ack =
+    topicProfix + getConfiger().configerMec.configerTopic.Topic_Camera_InterExter_Param_Query_Ack;
+m_MqttClientConfig.configerMec.configerTopic.Topic_Camera_InterExter_Param_Up_Ack =
+    topicProfix + getConfiger().configerMec.configerTopic.Topic_Camera_InterExter_Param_Up_Ack;
```

订阅列表也跟着换成新名字：

```
-...mqttSubscribeTopics[...] = strdup(topicConfiger.Topic_InternalExternalParams_Up_Ack.c_str());
-...mqttSubscribeTopics[...] = strdup(topicConfiger.Topic_InternalExternalParams_Query_Down.c_str());
+...mqttSubscribeTopics[...] =
strdup(m_MqttClientConfig.configerMec.configerTopic.Topic_Camera_InterExter_Param_Query_Ack.c_str());
+...mqttSubscribeTopics[...] =
strdup(m_MqttClientConfig.configerMec.configerTopic.Topic_Camera_InterExter_Param_Up_Ack.c_str());
```

注意命名也改了：`InternalExternalParams` → `Camera_InterExter_Param`，统一到协议文档的写法。

能学到什么

- **一个 topic 只能有一个"权威定义处"**。这里同一组 topic 在相机模块和运维模块各有一份， 谁都能改，改完就对不上。这是配置类问题最常见的根因。
- 判断方法：**"发的和收的，前缀是不是同一个来源？** " 只要前缀来自两个不同的配置对象， 迟早会错开。
- 这条和 `traffic_flow` 那份案例集里的 4-1（`LaneNO` / `LaneNo`）、4-4（`spatial_output` 挂错层级） 是同一个病：**协议字段/配置项没有单一来源。**

> **面试可以这么说**：设备接入里最容易错的是"topic 在哪定义、在哪用"。我们有过一次相机内外参查不到， 根因是这组 topic 定义在相机模块的配置里，但实际收发在运维模块，两边前缀不一样，订阅的名字和平台发的对不上。 后来统一挪到运维模块，用一套前缀。这类问题我的判断方法是看发布和订阅的前缀是不是同一个配置对象出来的。

2-3 相机接入模块：把没人用的配置项删掉

commit：`de23348`（2025-07-08，changxuhui）· ipcamera 模块 + 工厂类

改动

`StandardIpCamera::Init()` 里删掉了两个配置项：

```
-    frame_id_ = config["frame_id"].as<std::string>("camera_frame");
     reconnect_interval_ = config["reconnect_interval"].as<int>(5);
-    buffer_size_ = config["buffer_size"].as<int>(1024000);
```

为什么值得记一笔

这两个字段**读进来了但从来没用过**：

- `frame_id_`：原来大概是想给每路流一个坐标系名字，但下游用的是设备配置里的 frame_id，这个字段就废了。

- `buffer_size_`：读了个 1MB 的缓冲区大小，但实际拉流用的是 FFmpeg 的默认缓冲，这个值没传给任何地方。

"配置里读了一个值，但从来没传给用它的地方" —— 这是配置类 bug 的另一种形态： 不是"读错了"，而是"读了但没用"。它比读错更隐蔽，因为**配置文件里明明有这个键，运维会以为它生效。**

> **能学到什么**

- 加配置项时要问一句：**这个值最终传给了谁？** 如果答不上来，就是没接上。
- 反过来，清理配置项时也要连配置文件里的键一起删 —— 否则运维还在改一个死键。
- 这条在 `traffic_flow` 那份里也有同型案例（6-3 删旧管道时连带删配置字段）。

第三部分：设备与平台运维（连接、心跳、上报）

对应简历：「通过 MQTT 与运维平台交互，把设备状态、故障、版本信息按协议上报」。 **这一部分是整个 airos 里最有料的地方** —— 现场部署后出的问题，全在这儿。

3-1 运维云"连接还在、日志说连不上、运维掉线" —— 三个原因叠在一起

commit：`64e1682`（2026-04-15，zhangenwei）· BUG25800 · `om_component.cc` + `ccindex_component.cc`

现象

现场反馈：运维平台上这个路口的设备**显示离线**，但设备上 `ps` 看进程活着， 日志里反复打印 `[cloud][error]Connection is lost, reconnect cloud!`。 更奇怪的是**有时候连上了又掉，来回跳。**

原因一：用轮询判断连接状态，故障时把重连打成"自杀"

原来的做法是起一个定时器，每隔几秒轮一次：

```
bool CCINDEX_COMPONENT::monitorMqttConnectStatus()
{
    bool mqttConnected = false;
    bool mqttConnectedCloud = false;

    if (MQTTAsync_isConnected(m_MqttClient)) { mqttConnected = true; }
    else { mqttConnected = false; }

    if (MQTTAsync_isConnected(m_MqttClientCloud)) { mqttConnectedCloud = true; }
    else { mqttConnectedCloud = false; }

    if (!mqttConnectedCloud) {
        if (m_MqttClientCloud) {
            mqttReconnectCloud();          // ← 每次轮询发现"没连"就重连
        } else {
            mqttInitCloud();
        }
    }
    if (!mqttConnected) {
        if (m_MqttClient) { mqttReconnect(); } else { mqttInit(); }
    }
    return true;
}
```

问题在于 `MQTTAsync_isConnected()` **返回的是 Paho 客户端的本地状态**，不是链路真实状态：

- 服务端已经断开，但本地状态要等 keepalive 超时才翻过来 → 这期间一直返回 true（**假在线**）
- 反过来，客户端正在重连的过程中，本地状态是 false → **每次轮询都会再调一次** `mqttReconnect()`
- 重连是个异步过程，要好几百毫秒。下一个轮询周期又来了，发现还是 false，**又重连一次** → 把正在进行的连接打断，永远连不上

这就是"连接还在、日志说连不上"的来源：**日志是轮询打出来的，不代表真实状态；而反复重连让它真的连不上。**

原因二：重连成功了，但没有重新订阅

MQTT 的语义是：**连接断了，订阅关系就没了。**重连成功（TCP 层通了）不等于订阅恢复了。 Paho 的 `automaticReconnect` 或者 `MQTTAsync_connect` 成功后，必须重新 `subscribe`。

原来的代码只重连、不重订阅 → **"连上了，但一条指令都收不到"**。 运维平台那边看到设备在线（TCP 通），但下发什么都不响应 → 判定掉线。

原因三：clientId 不唯一，两个客户端互相踢

```
-    std::string clientIdTemp = m_MqttClientConfig.configerMqtt.mqttClientId + ":" + ...;
+    std::string clientIdTemp = m_MqttClientConfig.rscuEsn + m_MqttClientConfig.configerMqtt.mqttClientId + ":" +
...;
```

MQTT 规定 **同一个 clientId 只能有一个连接。**两个客户端用同一个 clientId 连同一个 broker 时， **broker 会把先连的踢掉**（后连的顶替）。

原配置如果是从模板复制过来的，多个设备可能是同一个 `mqttClientId` → 互相踢 → **反复上下线。** 加上 `rscuEsn`（设备唯一序列号）之后才真正唯一。

改法：从"轮询"改成"事件回调"

```
+std::atomic<bool> m_MqttConnected{false};
+std::atomic<bool> m_MqttConnectedCloud{false};

// 注册连接成功回调（首次连接和重连成功都会触发）
+MQTTAsync_setConnected(m_MqttClient, this, mqttConnectedCallback);
+MQTTAsync_setConnected(m_MqttClientCloud, this, mqttConnectedCallbackCloud);

+void CCINDEX_COMPONENT::mqttConnectedCallback(void* context, char* cause)
+{
+    auto thiz = (CCINDEX_COMPONENT *) context;
+    thiz->m_MqttConnected = true;
+    CCINDEX_INTER_DEBUG_PRINT << "[success]mqtt-inter (re)connected!";
+
+    // 关键：重连成功后必须重新订阅
+    MQTTAsync_responseOptions opts = MQTTAsync_responseOptions_initializer;
+    opts.onSuccess = mqttOnSubscribe;
+    opts.onFailure = mqttOnSubscribeFailure;
+    opts.context = thiz;
+    if (thiz->m_MqttClientConfig.configerMqtt.mqttRealSubscribeTopicNum > 0) {
+        MQTTAsync_subscribeMany(thiz->m_MqttClient,
+                                thiz->m_MqttClientConfig.configerMqtt.mqttRealSubscribeTopicNum,
+                                thiz->m_MqttClientConfig.configerMqtt.mqttSubscribeTopics,
+                                thiz->m_MqttClientConfig.configerMqtt.mqttSubscribeQoss,
+                                &opts);
+    }
+}
```

同时把整个轮询函数 `monitorMqttConnectStatus()` **整段注释掉**，状态改成 `std::atomic<bool>`。

能学到什么

- **连接状态不要轮询，用库提供的事件回调。**轮询在"正常"时看不出问题，一旦出故障， 它会变成**故障放大器**：越检测越坏。

- **"能连上"和"能收到消息"是两件事。**MQTT / WebSocket / 长连接都一样： 重连成功后订阅关系要重建。这一条是现场部署最高频的坑之一。
- **clientId 必须全局唯一**，而且要包含设备唯一标识。从模板复制的配置最容易犯这个错。
- **跨线程共享的状态用** `std::atomic`，不要用裸 `bool`。这里原来的 `bool` 被 MQTT 回调线程和 定时器线程同时读写，属于数据竞争（UB）。

> **面试可以这么说**：设备接入里最麻烦的是连接状态判断。我们原来是用定时器轮询 `MQTTAsync_isConnected`，发现没连就调重连。这个做法的坑是：一旦断线，每次轮询都会再发起一次重连， 把正在进行的连接打断，反而永远连不上；而且日志里看到的"连接丢失"是轮询打出来的，不代表真实状态。 后来改成用库的 connected 回调：回调里置状态位，并且**重新订阅所有主题**——因为 MQTT 重连后订阅关系是丢的， 不重订阅就是"连上了但收不到指令"，平台侧看就是在掉线。另外我们的 clientId 加了设备序列号， 原来多个设备用同一个 clientId 会被 broker 互相踢下线。

3-2 一个 channel 装下所有自检数据 —— 关键监控数据被挤丢

commit：`1b0b4b5`（2026-03-16，zhangenwei）· 14 个文件 · +1952/-593

现象

MEC 自检（设备自我体检）上报的数据**时有时无**，尤其是点云、雷达这几项，经常整项缺失。 自检本身没报错，就是数据不全。

原来的做法

所有自检项（SPAT 信号源、RASP 感知、CCIndex 交通指标、点云、雷达、各链路状态……） **都从同一个 CyberRT channel 读**：

```
// 所有自检项共用这一个 reader 配置
node_->CreateReader<airos::monitor_mec::MonitorMec>(
    m_MqttClientConfig.configerMec.configerMonitorMec.channel_readers, ...);
```

而且 channel 名字也不统一：`/om/spat/check`、`/v2x/monitor/spat`……

为什么错

一个 channel 里装的消息体 = 所有自检项的并集。问题有三层：

1. **消息太大**：一条消息要序列化全部自检项（含点云统计、雷达信息），单条消息体很大。 CyberRT 传输时对单条消息有大小/频率限制，超了会**丢整条。**
2. **节拍被最慢的拖住**：自检项里有的 1 秒一次（链路状态），有的 5 分钟一次（统计数据）。 塞在一条消息里，要么按快的发（大部分字段是旧值），要么按慢的发（快的项超时）。
3. **一个 reader 处理所有类型**：回调里靠 `has_md_xxx()` 一个个判断， 丢一条消息 = **所有项一起丢一次。**所以"关键数据丢失"是所有项同时缺，不是随机缺。

改法：拆成 10 个 channel，每项一个

```
// 拆开后，每项一个独立的 reader + 独立的 channel 名
initProcessMecCheckSpatSrcData();   // /v2x/mec/om/check/spat_src/data_
initProcessMecCheckRsapData();      // /v2x/mec/om/check/rsap/data_
initProcessMecCheckRsapLink();      // /v2x/mec/om/check/rsap/link_
initProcessMecCheckCcindexV2xData();// /v2x/mec/om/check/ccindex/...
initProcessMecCheckTmData();        // ...
initProcessMecCheckTmLink();        // ...
initProcessMecCheckStData();        // ...
initProcessMecCheckStLink();        // ...
initProcessMecCheckCcindexRcpData();// 点云/雷达
initProcessMecCheckCcindexRcpLink();
```

每个配置项长这样（`A_` / `B_` 前缀是序列化顺序号）：

```
std::string channel_readers = "/v2x/mec/om/check/spat_src/data_";
JsonSerialize(channel_readers, "A_channel_readers", j, false);
```

命名也统一了：全部挂到 `/v2x/mec/om/` 下面，`data_` / `link_` 后缀区分数据类还是链路类。

能学到什么

- **"一个大消息装所有东西"是设计问题，不是调参能救的。**它一定会同时踩三个坑： 消息过大、节拍被迫统一、丢一次全丢。
- 拆的时候**按"生命周期/频率"拆，不是按"逻辑模块"拆。**这里 `data_` 和 `link_` 分开， 就是因为数据类按业务周期（1s ~ 5min），链路类按心跳周期（秒级），完全不同的节奏。
- **channel / topic 命名要带命名空间前缀。**原来 `/om/spat/check` 这种名字放到全局命名空间里， 和别的模块撞名的概率很高；统一到 `/v2x/mec/om/` 之后，一眼能看出归属。

> **面试可以这么说**：设备自检原来所有检测项都塞在一个 CyberRT channel 里发，现场发现监控数据经常整批缺失。 根因是消息体太大、而且各项频率差很多——链路状态是秒级，统计数据是五分钟级，塞一条消息里必然互相拖累， 丢一条就全丢。后来按"数据类 / 链路类"拆成十个独立 channel，命名统一加 `/v2x/mec/om/` 前缀。 这件事让我觉得，一个消息里装多个不同频率的指标，是很危险的设计。

3-3 拆 channel 拆到一半 —— 两个子项还共用一个

commit：`1b0b4b5`（同一提交里的残留）

现象

拆成 10 个 channel 之后，点云的"数据"和"链路"这两项**仍然共用一个 channel**：

```
void OM_COMPONENT::initProcessMecCheckCcindexRcpData()
{
    auto reader = node_->CreateReader<airos::monitor_mec::MonitorMec>(
            m_MqttClientConfig.configerMec.configerMonitorMecRcp.channel_readers,   // ← Rcp
            std::bind(&OM_COMPONENT::processMecCheckCcindexRcpData, this, std::placeholders::_1));
}

void OM_COMPONENT::initProcessMecCheckCcindexRcpLink()
{
    auto reader = node_->CreateReader<airos::monitor_mec::MonitorMec>(
            m_MqttClientConfig.configerMec.configerMonitorMecRcp.channel_readers,   // ← 还是 Rcp
            std::bind(&OM_COMPONENT::processMecCheckCcindexRcpLink, this, std::placeholders::_1));
}
```

其他 8 项都是一项一个配置（`configerMonitorMecSpatSrcData`、`...RsapData`、`...RsapLink`……），**只有点云这一对用了同一个 `configerMonitorMecRcp`**。

为什么这是个隐患

CyberRT 里同一个 channel 可以有多个 reader，两个都能收到。所以**消息不会丢**，但会：

- 每个 reader 都收到全部消息，各自靠 `has_md_radar_cp()` / `has_ml_radar_cp()` 判断 —— 等于把"拆 channel"省下的带宽又浪费回去了
- 如果以后有人给 `Rcp` 这个配置改成"只发数据类"，链路项就静默失效

而且同一段代码里，`processMecCheckCcindexRcpData` 的更新调用是**注释掉的**：

```
// handleMonitorDataUpdate(std::ref(m_MonitorStateCcindexRcp),
//     mecDeviceData->md_radar_cp().tag(), mecDeviceData->md_radar_cp().timestamp());
```

也就是说**雷达原始数据这一项当时根本没在检测**，只检测了链路。属于"拆到一半先提交"的状态。

能学到什么

- **大重构里最容易漏的是"最后一个"**。10 项拆 9 项，剩下的那对共用，而且**不会报错。** 拆完必须回头数一遍：**N 个 reader 对应 N 个配置项吗?**
- 半成品提交是常态（能跑、不崩、主流程 OK 就先上）。但**必须留 TODO 或者 issue 号**，否则半年后没人知道"雷达数据项没检测"是 bug 还是设计。
- 读别人代码时，看到 `// handleXxx(...)` 被注释掉，要立刻警觉：**这块功能当前是关的。**

> **面试可以这么说**：我读那段拆 channel 的提交时发现，十项里九项都拆成独立配置了，只有点云的数据和链路 还共用一个，而且数据项的处理调用被注释掉了——等于当时只有链路在自检。这说明大重构经常是"拆到能跑为止"，剩下的一致性检查没人做。所以我后来看这类重构，都会数一遍 reader 数量配不配得上配置项数量。

3-4 所有自检项的超时阈值一律拍 3 秒 —— 慢周期数据永远超时

commit：`6daadbe`（2026-04-03，zhangenwei）· `configer_om_mec.h`

现象

MEC 自检里"定时统计数据""定时评价数据""行人非机动车数据"这几项**长期告警**， 但上游其实一直在正常发数据。

原来的写法

`MecSelfCheckTime` 构造函数里，超时阈值几乎全是 3 秒：

```
ccindex_tm_data_statistics_tr.timeoutSecond = 3;      // 定时统计数据
ccindex_tm_data_statistics_tr.recoverySecond = 3;
ccindex_tm_data_evaluations_tr.timeoutSecond = 3;     // 定时评价数据
ccindex_tm_data_evaluations_tr.recoverySecond = 3;
ccindex_tm_data_nonmotor_tr.timeoutSecond = 40;       // 行人非机动车
ccindex_tm_data_nonmotor_tr.recoverySecond = 40;
```

为什么错

超时阈值必须大于数据的真实发布周期。这几个数据的周期是：

数据	真实周期	原阈值	结果
实时轨迹 / 排队 / 溢出 / 出口通道	1 ~ 3 秒	3 秒	勉强够
定时统计数据	**5 分钟**（300 秒）	3 秒	**每个周期里 99% 的时间都在告警**
定时评价数据	**1 分钟**	3 秒	同上
行人非机动车	约 45 秒	40 秒	边界上反复抖

3 秒的阈值对 5 分钟一次的数据来说，等于**永远处于"超时"状态。**告警一直在，恢复一直在， 平台侧的告警列表被刷满，真正的问题反而看不见。

改法：按真实周期重新标定

```
-ccindex_tm_data_statistics_tr.timeoutSecond = 3;
-ccindex_tm_data_statistics_tr.recoverySecond = 3;
+ccindex_tm_data_statistics_tr.timeoutSecond = 320;      // 300s 周期 + 20s 余量
+ccindex_tm_data_statistics_tr.recoverySecond = 320;

-ccindex_tm_data_evaluations_tr.timeoutSecond = 3;
-ccindex_tm_data_evaluations_tr.recoverySecond = 3;
+ccindex_tm_data_evaluations_tr.timeoutSecond = 65;      // 60s 周期 + 5s 余量
+ccindex_tm_data_evaluations_tr.recoverySecond = 65;

-ccindex_tm_data_nonmotor_tr.timeoutSecond = 40;
-ccindex_tm_data_nonmotor_tr.recoverySecond = 40;
+ccindex_tm_data_nonmotor_tr.timeoutSecond = 45;
+ccindex_tm_data_nonmotor_tr.recoverySecond = 45;
```

注意 `timeoutSecond` 和 `recoverySecond` 是**成对改**的：一个管"多久没数据算异常"， 一个管"恢复持续多久才算真恢复"。两个都按周期标，否则会出现"秒级抖动的告警风暴"。

能学到什么

- **超时阈值不是拍脑袋的数字，它必须由数据周期推导**：`阈值 = 周期 + 余量`。 余量一般取周期的 10% 或固定几秒，看链路抖动情况。
- **"告警一直在"本身就是个 bug 信号**。持续告警会让运维对告警脱敏 —— 这比不告警更危险，因为它训练人忽略告警。
- 这类问题在代码里看是"一个数字"，但根因是**设计时没有把"数据周期"当成配置的一部分**。 更彻底的做法是：让每个检测项**自带它的预期周期**，超时阈值 = 周期 × 系数，自动算出来。

> **面试可以这么说**：设备自检里有十几项，每项配一个超时阈值。原来几乎全是 3 秒，但其中几项的数据是 五分钟才发一次，等于一直处于超时状态，告警列表被刷满，真正的问题反而看不见。后来按实际周期重新标定， 五分钟的给 320 秒，一分钟的给 65 秒。这件事让我觉得，超时阈值必须从数据周期推导出来， 不能所有项拍同一个数——而且"告警一直在响"本身就是个 bug，它会让运维对告警脱敏。

第四部分：进程退出与多线程

这一部分和简历三条都相关 —— **设备服务是常驻进程，退出/重启时的行为直接决定现场能不能正常升级。** `994ac52` 这一个提交里就有四个经典问题，值得逐条看。

4-1 MQTT 回调返回 0 —— broker 会重投，但消息已经被 free 了

commit：`994ac52`（2026-09-10，libo）·「收到非法 JSON 时 double free」

现象

设备收到某条格式错误的 JSON 后，进程**直接崩溃**（double free / glibc 报 `free(): double free detected in tcache 2`），而且**崩得很有规律**：每次都崩在同一类坏消息上。

原来的写法

```
int CCINDEX_COMPONENT::mqttSubscribeMsgArrvd(void *context, char *topicName,
                                             int topicLen, MQTTAsync_message *message)
{
    int ret = 1;
    // ...
    try {
        trafficMetricsJson = afl::base::json::parse(
            (char *) message->payload,
            (char *) message->payload + message->payloadlen);
    }
    catch (...) {
        MQTTAsync_freeMessage(&message);      // ① 释放消息
        MQTTAsync_free(topicName);            // ② 释放 topic
        RADAR_TRAFFIC_METRICS_ERROR_PRINT << "[error]mqtt-inter parse json failure!";
        return false;                          // ③ 返回 0
    }
}
```

为什么错：Paho 的回调返回值有语义

`messageArrived` 回调的返回值决定 broker 怎么处理这条消息：

返回值	含义
`1`	已消费，确认收到，broker 不再重投
`0`	**处理失败，broker 会重投同一条消息**

`return false` 在 C++ 里就是 `return 0`。于是流程变成：

1. 收到坏 JSON → catch → `freeMessage(&message)`（**message 指针已被释放**）
2. 返回 0 → Paho 认为失败 → **拿同一个 `message` 指针重新投递**
3. 回调再次被调用，`message` 指向已释放内存 → 读 `message->payloadlen` 就是 use-after-free
4. 再一次 `freeMessage(&message)` → **double free**

关键点：返回 0 时，释放权归 paho，你不该自己 free；如果自己 free 了，就必须返回 1。

改法

```
-    catch (...) {
+    catch (const std::exception& e) {
+        // 坏消息直接丢弃：释放后必须返回1。返回0会让paho用已释放的指针重投，导致double free
+        RADAR_TRAFFIC_METRICS_ERROR_PRINT
+            << "[error]mqtt-inter " << "parse json failure!"
+            << " [topic]" << topic
+            << " [payloadlen]" << message->payloadlen
+            << " [what]" << escapeForLog(e.what(), strlen(e.what()))
+            << " [payload]" << escapeForLog(message->payload, message->payloadlen);
         MQTTAsync_freeMessage(&message);
         MQTTAsync_free(topicName);
-        return false;
+        return 1;
     }
```

同一个提交里还有一处更隐蔽的：

```
     if(topicProfix != "upload") {
         CCINDEX_CLOUD_DEBUG_PRINT << "[error]mqtt-cloud topic profix is not upload!";
-        return -1;                              // ← -1 也不是 1，同样会重投
+        MQTTAsync_freeMessage(&message);
+        MQTTAsync_free(topicName);
+        return 1;
     }
```

`return -1` **和** `return 0` **一样，都不是 1** —— 只要不是 1，paho 就重投。

能学到什么

- **回调的返回值是有语义的，不是"成功/失败"这么简单。**用第三方库之前必须看清： 返回值怎么解释、谁负责释放内存。这里"返回 0 = 请你再投一次"是 paho 的约定。
- `return false` **在 int 返回值的函数里就是** `return 0`。C++ 允许 bool→int 隐式转换， 编译器不会警告。所以回调里**不要用** `false` / `true`，**直接写** `0` / `1`，语义更清楚。
- **释放内存和"告诉对方已处理"必须一致**：自己 free 了就一定要返回 1，否则对方拿野指针再来一次。
- 这条的排查线索很典型：**崩溃固定发生在"收到某些特定消息"时**，说明和消息内容相关； 再看是不是在错误分支里 free 过。

> **面试可以这么说**：我们用 Paho 的 MQTT 客户端，有个双击崩溃的问题，专门在某些格式错误的报文上必现。根因是回调里解析 JSON 失败后，我们先 free 了消息，然后 `return false` ——而 Paho 的约定是返回 0 表示处理失败、请重投这条消息，于是它拿已经释放的指针又投了一次，第二次 free 就 double free。 修法就是释放之后必须返回 1，明确告诉它"这条我丢了，别再给我"。这件事让我记住：用回调式库， 返回值语义和内存归属必须一起看。

4-2 MQTT 的 payload 不是以 `\0` 结尾的

commit：`994ac52`（同一提交）

原来的写法

```
CCINDEX_CLOUD_DEBUG_PRINT << "[notice]cloud >>> ccindex"
    << " [topic]" << topicName
    << "[msg]" << (char *) (message->payload);       // ← 当 C 字符串用
```

为什么错

MQTT 的 payload 是**二进制的、带长度（`payloadlen`）、不保证以 `\0` 结尾。** `(char*)payload` 交给 `operator<<` 时，它会一直读到遇到 `\0` 为止 —— **越过 payload 边界读到堆上的其他数据。**

后果分两种：

- 运气好：后面正好有个 `\0`，打印出来的日志末尾带一串垃圾字符
- 运气差：一直读越界，**段错误**或者把敏感内存打进日志

同理，`std::string s(payload)` 也是错的，必须用带长度的构造：

```
+    << "[msg]" << std::string((char *) message->payload, message->payloadlen);
```

附带：一个专门用来打非法报文的工具函数

同一提交加了 `escapeForLog()`：

```
// 把字节串（MQTT payload、json异常信息）转成可打印字符串用于日志：可见ASCII原样输出，
// 其它字节（含反斜杠）转成\xHH，这样非法UTF-8、控制字符、截断等都能看出来，也不会把日志文件变成二进制；
// 过长时只保留头尾
static std::string escapeForLog(const void* data, int len)
{
    const int kHead = 1024;
    const int kTail = 512;
    std::string out;
    const unsigned char* p = static_cast<const unsigned char*>(data);
    if (p == nullptr || len <= 0) return out;
    auto append = [&out, p](int begin, int end) {
        char hex[8];
        for (int i = begin; i < end; ++i) {
            if (p[i] >= 0x20 && p[i] < 0x7f && p[i] != '\\') out += static_cast<char>(p[i]);
            else { snprintf(hex, sizeof(hex), "\\x%02x", p[i]); out += hex; }
        }
    };
    if (len <= kHead + kTail) append(0, len);
    else {
        append(0, kHead);
        out += "...(" + std::to_string(len - kHead - kTail) + " bytes omitted)...";
        append(len - kTail, len);
    }
    return out;
}
```

这个函数解决了三个问题：

1. **非法字节可见**：`\xHH` 转义后，能看出是截断、控制字符还是编码问题
2. **日志不会变二进制**：直接往里写非法 UTF-8，日志文件会被 `grep` 当二进制，`grep -a` 才能搜
3. **不会刷爆**：只留头 1024 + 尾 512 字节，中间写明省略多少

能学到什么

- **凡是"带长度的二进制数据"，一律不要当 C 字符串用。**C++ 里用 `std::string(ptr, len)`， C 里用 `memcpy` + 自己加 `\0`。
- **排障工具要"能看出坏在哪"。**原来日志只打一句 `parse json failure!`， 根本不知道坏消息长什么样；加上 payload 转义后，一眼能看出是 `\x00` 截断还是 UTF-8 断裂。
- 打印外部数据必须**限长。**不然一条 10MB 的坏报文能把日志盘写满。

> **面试可以这么说**： MQTT 的 payload 是带长度的二进制，不保证以 0 结尾。我们原来日志里直接 `(char*)payload` 当字符串打，会越界读到后面的内存，偶尔还会崩。后来改成带长度的 string 构造。 同时加了个转义函数，把不可打印字节打成 `\xHH`，只留头尾各一千字节——这样坏报文能看出坏在哪， 日志也不会被 grep 当二进制文件。

4-3 EventLoop 只能在创建它的线程里析构 —— 所以故意泄漏

commit： `994ac52`（同一提交）

现象

进程正常退出（`systemctl stop` 或者升级重启）时**段错误**， 栈在 `~EventLoop()` 里的 `assertInLoopThread`。

为什么错

这个项目用的 `afl::net::EventLoop`（muduo 风格）有一条硬约束：**`~EventLoop()` 里会检查 "当前线程是不是创建它的那个线程"，不是就 assert 失败。**

而组件的析构是在**主线程**执行的，EventLoop 却是在各个 worker 线程里 `make_shared` 出来的。 进程退出时：

1. 主线程走组件析构
2. 组件里的 `shared_ptr<EventLoop>` 引用计数归零 → 在主线程执行 `~EventLoop()`
3. `~EventLoop()` 发现当前线程不是 loop 线程 → assert / 崩溃

改法： `leakAtExit()` —— 有意的内存泄漏

```
// EventLoop只能在创建它的线程里析构（~EventLoop里会assertInLoopThread），线程退出后已无法满足。
// 组件只在进程退出时才析构，所以EventLoop和挂在上面的UDP Channel有意不释放，交给进程退出回收
static void leakAtExit(std::shared_ptr<afl::net::EventLoop>& loop)
{
    new std::shared_ptr<afl::net::EventLoop>(std::move(loop));
}
```

把 `shared_ptr` **move 到一个 new 出来的、永不 delete 的 `shared_ptr` 里**， 引用计数永远不归零， `~EventLoop()` 永远不会被调用，交给 OS 在进程退出时整体回收。

调用处：

```
    // 4. EventLoop和UDP Channel不在这里析构，见leakAtExit的说明
    for (auto* loop : loops) {
        leakAtExit(*loop);
    }
    (void)m_UdpChnl.release();       // UDP Channel 同理，release 掉不 delete
```

为什么这是"对的做法"而不是"偷懒"

- 进程退出后，OS 会回收**所有**内存和 fd，泄漏在退出阶段**没有实际代价**
- 反过来，硬要在退出阶段按正确顺序析构，要处理：线程 join 顺序、定时器注销、 callback 里的临时对象、EventLoop 的线程亲和性……每一条都可能引入新的崩溃
- **"退出阶段故意泄漏"是成熟项目里常见的手法**（Chromium、Node.js 退出时都有类似处理）

前提是：**泄漏只发生在进程生命周期的最后一步**，且**注释写清楚为什么**。这里注释做到了。

能学到什么

- **线程亲和性的对象（EventLoop、某些 GUI 对象）不能跨线程析构。** 这类对象要么在它自己的线程里销毁（投递一个"删除自己"的任务），要么就别销毁。
- **"故意泄漏"要满足三个条件**才不是耍赖： ① 只在退出路径上；② 有明确注释说明原因；③ 泄漏量固定、不随运行时间增长。
- 排查线索：**只在退出时崩、运行期完全正常** → 八成是析构顺序或者线程亲和性问题。

> **面试可以这么说**：我们进程正常退出时会崩，栈在 EventLoop 的析构里。原因是这类网络库的 EventLoop 只能在创建它的线程里析构，而组件析构是在主线程做的。我们最后没去硬解这个顺序问题，而是把 EventLoop 故意泄漏掉，交给进程退出回收——因为退出阶段泄漏没有实际代价，硬要按顺序析构反而容易引入新崩溃。 前提是只在退出路径上泄漏，而且注释写清楚为什么。

4-4 启动顺序：EventLoop 还没建好，主流程已经开始用了

commit：`994ac52`（同一提交）

现象

启动时偶发空指针崩溃（`m_Eventloop` 为 null），**不是每次都崩**，负载高的时候更容易复现。

原来的写法

```
bool CCINDEX_COMPONENT::Init()
{
    // 每个 EventLoop 在各自线程里创建
    m_Task.reset(new std::thread([&](){ initEventLoop(); }));
    m_TaskTrafficMetrics.reset(new std::thread([&](){ initEventLoopTrafficMetrics(); }));
    // ... 一共 7~9 个线程

    m_TaskUseCase.reset(new std::thread([&](){ initUseCase(); }));

    doBaiscWork();          // ← 立刻开始干活，可能马上用到还没创建的 EventLoop
    return true;
}
```

`std::thread` 构造完就返回了，**不代表线程里的函数已经执行到 `make_shared` 那一行。** `doBaiscWork()` 里如果马上注册定时器（`m_Eventloop->runAfter(...)`）或者发 MQTT 请求， 而 `m_Eventloop` 还是空的 → 空指针。

"线程启动应该够快"是个假设，不是保证 —— 系统负载高、CPU 抢占时，线程可能几百毫秒才开始跑。

改法：用条件变量等"全部就绪"

```
// EventLoop构造时绑定当前线程，只能在各自线程里创建；这里等上面7个EventLoop都创建好，
// 避免doBaiscWork()和随后到达的MQTT回调访问到还没创建的EventLoop（空指针）
waitEventLoopsReady(7);
doBaiscWork();
```

```
void CCINDEX_COMPONENT::notifyEventLoopReady()
{
    std::lock_guard<std::mutex> lock(m_EventloopReadyMutex);
    ++m_EventloopReadyCount;
    m_EventloopReadyCv.notify_all();
}

void CCINDEX_COMPONENT::waitEventLoopsReady(int count)
{
    std::unique_lock<std::mutex> lock(m_EventloopReadyMutex);
    if (!m_EventloopReadyCv.wait_for(lock, std::chrono::seconds(10),
                                     [this, count]() { return m_EventloopReadyCount >= count; }))
    {
        CCINDEX_INTER_ERROR_PRINT << "[error]wait eventloop ready timeout!"
                                  << " [ready]" << m_EventloopReadyCount << " [expect]" << count;
        return;
    }
    // ...
}
```

每个 `initEventLoopXxx()` 里， `make_shared` 之后立刻 `notifyEventLoopReady()`：

```
bool CCINDEX_COMPONENT::initEventLoop()
{
    m_Eventloop = std::make_shared<afl::net::EventLoop>();
    // ...
    notifyEventLoopReady();
    m_Eventloop->loop();        // 阻塞在这里跑事件循环
    return true;
}
```

能学到什么

- **多线程初始化要有"就绪屏障"**，不能靠 sleep、也不能靠"应该够快"。 条件变量 + 计数器是最简单的实现。
- **`wait_for` 必须带超时，超时要打日志继续走**（不能用 `wait` 死等）—— 否则某个线程挂了，整个进程永远起不来，现场表现为"卡在启动"。这里给 10 秒并打 error。
- **`count` 是硬编码的魔法数字（7）**。更稳的写法是让线程数量由同一个来源决定， 或者每个线程注册完自己 notify，用"应到人数"和"实到人数"比对。硬编码 7 的风险是： 以后加了第 8 个 EventLoop，忘了改这里 → 启动直接超时 10 秒（虽然不崩，但慢）。

> **面试可以这么说**：我们的服务里有好几个 EventLoop，每个都必须在自己的线程里创建。原来是把线程起起来 就直接往下走，结果偶发空指针——因为线程创建完不代表里面的对象已经 new 出来了。后来加了个就绪屏障：每个 EventLoop 建好后给计数器加一，主线程用条件变量等计数到齐再继续。等待带了 10 秒超时， 超时打日志继续走，避免一个线程出问题导致整个进程卡在启动。

4-5 进程退出时，两个上传线程永远卡在 `wait` 上

commit：`994ac52`（同一提交）

原来的写法

```
bool CCINDEX_COMPONENT::uploadQueue1()
{
    while (true) {
        std::unique_lock<std::mutex> lock(cv_m1);
        cv1.wait(lock, [&] { return queue1_ready; });        // 只等这个标志
        // ... 处理上传
    }
}
```

为什么错

`shutdownWorkers()` 里要 `join` 这两个线程。但线程正卡在 `cv1.wait` 上， 而 `queue1_ready` 只有在"有数据要传"时才会被置 true。

退出时没有新数据 → `queue1_ready` 永远是 false → `wait` 永远不返回 → **`join` 永远等下去 → 进程退不出去。**

现场表现： `systemctl stop` 之后进程停不下来，要等 systemd 超时 `SIGKILL`。 如果这时候正在写文件/上报，可能留下半个文件。

改法：把"退出"也做成谓词的一个分支

```
-        cv1.wait(lock, [&] { return queue1_ready; });
+        cv1.wait(lock, [&] { return queue1_ready || m_StopWorkers; });
+        if (m_StopWorkers) {
+            return true;                 // 退出标志置了就直接返回，让 join 可以完成
+        }
```

`shutdownWorkers()` 里的配合：

```
    // 1. 置退出标志（之后MQTT回调直接丢弃新消息），让所有EventLoop退出loop()并join
    m_StopWorkers = true;
    for (auto* loop : loops) { if (*loop) (*loop)->quit(); }
    // ... join 所有 EventLoop 线程
    // 2. 唤醒并join上传队列线程
    {
        std::lock_guard<std::mutex> lock(cv_m1);     // ← 先拿锁
    }
    cv1.notify_all();                                 // ← 再通知
```

那个"空作用域加锁"不是废话：**它保证 `notify_all` 发生在锁释放之后**， 避免正在 `wait` 的线程被唤醒后立刻又抢不到锁、重新进入等待。

能学到什么

- **`wait(lock, pred)` 的谓词必须包含"退出"条件。**只写业务条件，退出时必然挂住。 正确形式永远是 `return business_ready || stopping;`，然后紧接着 `if (stopping) return;`。
- 这条和 `traffic_flow` 那份案例集的 **3-2（析构时消费者永远卡在 wait 上）是完全同一个坑** —— 两个仓库、两个作者、同样的错误。说明这不是"某人水平问题"，而是**这个写法的天然陷阱。**
- **`notify` 前要不要持锁**：持锁能避免"唤醒丢失"，但在锁外 notify 性能更好。 折中做法就是这里这样：**持锁进作用域再立刻释放，然后 notify。**

> **面试可以这么说**：我们的上传线程是"条件变量 + 队列"的生产消费模型，原来谓词只写了"队列有数据"。 退出的时候没有新数据，线程就一直卡在 wait 上，join 永远等不到，进程停不下来，最后被 systemd 强杀。 修法是把退出标志也放进谓词里： `return ready || stopping` ，醒来先判断是不是要退出。 这个坑其实很常见——我在另一个项目里见过一模一样的写法，说明只写业务条件的谓词天生有这个问题。

4-6 锁了对象，没锁容器 —— `std::map` 的 `operator[]` 不是线程安全的

commit：`4fea6d5`（2026-03-23）+ `f52f0ed`（2026-03-20），zhangenwei

现象

MEC 自检在多设备场景下偶发崩溃 / 数据错乱，core dump 的栈有时在 map 操作里，有时在 state 读写里。 **单设备测不出来，多设备压测才出。**

第一步：给状态对象加锁（`f52f0ed`）

每个检测项的状态是一个 `MonitorState` 结构体，被两个线程同时访问：

- MQTT 回调线程：收到数据 → 更新 `lastReceivedTime`、`isAlarmActive`
- 自检定时器线程：定期检查 → 读 `lastReceivedTime`、决定发告警

```
 struct MonitorState {
     // ...
+    mutable std::mutex stateMutex;
 };

+std::lock_guard<std::mutex> lock(state.stateMutex);
```

8 处读写全加锁。`mutable` 是因为有些方法在 `const` 成员函数里也要加锁。

第二步：发现还不够 —— 容器本身也要锁

`4fea6d5` 的标题是「针对动态数据和静态数据存在竞争，**细化锁的颗粒度**」，实际改的是：

```
+std::mutex m_CcindexTmMapMutex;      // 保护 m_MonitorStateCcindexTmMap
+std::mutex m_CcindexStMapMutex;      // 保护 m_MonitorStateCcindexStMap

// 插入/查找时
+std::lock_guard<std::mutex> mapLock(m_CcindexTmMapMutex);
```

为什么：状态对象是存在 `std::map` 里的，而且用的是 **`operator[]`**：

```
auto& state = m_MonitorStateCcindexRcpData[deviceId];   // ← 隐式插入
```

`operator[]` 在 key 不存在时会**插入新元素**，插入可能触发红黑树 rehash / 节点重排。 两个线程同时 `operator[]` 不同的 key → 树的指针被同时改 → **崩溃或数据丢失**。

锁住 value 是没用的 —— 你锁的 `state.stateMutex` 属于某个元素，而容器结构的修改根本不经过那把锁。

改法汇总

1. **容器级锁**：每个 map 一把锁，保护插入 / 查找 / 遍历
2. **元素级锁**：每个 state 一把锁，保护字段读写
3. **两层都不能省**：先 mapLock 拿到元素指针，再 stateLock 改字段，**赶紧释放 mapLock**（不要把 mapLock 一直持有到处理完 —— 那就成了一把大锁）

```
auto it = m_MonitorStateCcindexRcpData.find(deviceId);
if (it == m_MonitorStateCcindexRcpData.end()) return;
auto statePtr = it->second;                    // 拿到指针就放开容器锁
if (statePtr) {
    std::lock_guard<std::mutex> lock(statePtr->stateMutex);   // 再锁元素
    if (!statePtr->enable) return;
    // ... 处理
}
```

能学到什么

- **"我锁了对象"不等于"线程安全"**。要问三个层次： ① 容器结构（插入/删除/遍历）锁了吗？ ② 元素内容锁了吗？ ③ 指针/迭代器在拿到之后还有效吗（别人可能 erase）？

- **`map[key]` / `map.at(key)` / 遍历，读起来都像"只读"，但 `operator[]` 会写。** 并发场景里查 map 应该用 `find()`，它是纯读，不会插。
- **锁粒度要分层**：一把大锁最安全但性能差；只锁元素会崩；正确做法是"容器锁短暂持有 + 元素锁长期持有"。 这就是标题里"细化锁的颗粒度"的真实含义。

> **面试可以这么说**：我们自检的状态存在 map 里，每个状态对象有自己的锁。加了锁之后多设备压测还是崩， 原因是 map 用的是 `operator[]`，key 不存在时会插入，插入要改树结构，而这一步根本没走过元素锁。 所以后来分两层：容器一把锁管插入和查找，元素一把锁管字段，拿到元素指针就赶紧放开容器锁。 这件事让我意识到，"锁了对象"和"容器线程安全"是两件事， `operator[]` 看着像读其实是写。

第五部分：协议字段、单位、配置

这一类的共同点：**改一个字符，编译器、运行期、日志全都不吭声，只有对端知道错了。**

5-1 内存单位标成 MB，值其实是字节

commit：`0c0e685`（2026-08-27，root）· `performence_utils.cc`

现象

运维平台显示设备内存"总容量 8589934592 MB"这种荒谬数字，或者反过来，怎么算都对不上。

原来的写法

```
bool PerformenceUtils::getMemInfo(MemInfo &memInfo)
{
    // ...
    memInfo.total = (double) totalMem * ByteUnit;
    memInfo.used  = (double) usedMem  * ByteUnit;
    memInfo.free  = (double) availableMem * ByteUnit;
    if(printInfo) {
        ss << std::left << std::setw(20) << "mem-total:" << memInfo.total << "MB" << std::endl;
        ss << std::left << std::setw(20) << "mem-used:"  << memInfo.used  << "MB" << std::endl;
        ss << std::left << std::setw(20) << "mem-free:"  << memInfo.free  << "MB" << std::endl;
    }
}
```

为什么错

`ByteUnit` 是个换算系数。如果它是 `1024 * 1024`，那结果确实是 MB；如果是 `1`（很多系统 `/proc/meminfo` 直接给 KB，乘 1024 得字节），那结果就是**字节**。

这里的实际结果：`memInfo.total` 是**字节**（因为 `/proc/meminfo` 的 `MemTotal` 是 KB， 乘 1024 得到字节），但打印时硬写了 `"MB"`。

数值是对的，单位标签是错的。 而运维平台是按标签解析的 —— 它看到 "8589934592 MB"， 要么显示荒谬数字，要么解析失败。

改法

把标签改成实际的单位：

```
-       ss << ... << memInfo.total << "MB" << std::endl;
+       ss << ... << memInfo.total << "B"  << std::endl;
-       ss << ... << memInfo.used  << "MB" << std::endl;
+       ss << ... << memInfo.used  << "B"  << std::endl;
-       ss << ... << memInfo.free  << "MB" << std::endl;
+       ss << ... << memInfo.free  << "B"  << std::endl;
```

能学到什么

- **"数值 + 单位"必须当成一对来维护。**只改数值算法、不改单位标签，是最容易漏的一步 —— 因为它们在代码里是两个分开的东西：一个是变量，一个是字符串字面量。
- 更彻底的写法是**让数值类型携带单位**（比如返回值就叫 `totalBytes`）， 或者提供 `formatBytes()` 统一换算成人类可读的 `1.2 GB`，**不要让调用方自己拼单位。**
- 这和 `traffic_flow` 那份的 **1-8（车尾时刻换算错 12.96 倍）** 是同一类： 都是"单位和数值没对齐"，只是一个错在换算、一个错在标签。
- 排查线索：**运维平台上的数字"量级不对"**（差 1024 倍、差 1024² 倍）→ 九成是单位问题， 而且往往只有一处：标签。

> **面试可以这么说**：我们上报设备内存，原来打印的时候写死了 "MB"，但算出来的值其实是字节， 运维平台上就显示成一个天文数字。数值逻辑没问题，就是单位标签错了。这件事让我觉得， 数值和单位应该绑定在一起——变量名带单位，或者提供一个统一的格式化函数， 不要让每个打印点自己拼单位字符串。

5-2 `timeStamp` 和 `timestamp` —— 一个大小写，整个字段的值全丢

commit：`8c14386`（2026-04-29，hezhiyong）· 5 处

现象

OTA 升级取消的应答消息，运维平台收到的 `timestamp` 全是 0 / 空，平台判定为无效消息， 取消升级永远不成功。

原来的写法

```
OtaUpdateCancelAckData otaUpdateCancelAckData;
-otaUpdateCancelAckData.timeStamp = afl::util::TimeStamp::now(true).millSeconds();
+otaUpdateCancelAckData.timestamp = afl::util::TimeStamp::now(true).millSeconds();
otaUpdateCancelAckData.seqNum = otaUpdateCancelData.seqNum;
otaUpdateCancelAckData.deviceID = otaUpdateCancelData.rscuEsn;
otaUpdateCancelAckData.state = "received";
```

为什么能编过

能编过，说明结构体里真的存在 `timeStamp` 这个成员 —— 它和 `timestamp` 是两个不同的字段。 大概率是：

- 老版本用 `timeStamp`（驼峰），后来协议改成 `timestamp`（全小写），新字段加上了，老字段没删
- 或者基类里有个 `timeStamp`，派生类里又有个 `timestamp`

于是 `otaUpdateCancelAckData.timeStamp = ...` 写的是一个**不会被序列化出去的字段**（或者序列化成另一个 JSON key），真正上报的 `timestamp` 永远是默认值 0。

同一份代码里 5 处都写成了 `timeStamp` —— 说明是从一个地方复制出来的， 没人对过协议文档。

改法

5 处全部改成 `timestamp`。

能学到什么

- **"编译通过"不代表"字段对了"**。结构体里留着两个只差大小写的字段时，编译器完全帮不了你。
- **协议字段名要以协议文档为唯一权威**，而且**改协议时要把旧字段删掉。** 旧字段留着不删就是一个随时会踩的陷阱——它躺在那里，IDE 自动补全还会优先弹出它。
- 排查线索：**"字段有值但对面收到 0/空"** → 第一反应查字段名拼写（特别是大小写和下划线）。 JSON 序列化是最容易暴露这个的：serialize 里没这个 key，或者 key 名字不一样。
- 这和 `traffic_flow` 那份的 **4-1（`LaneNO` 写成 `LaneNo`）** 是同一类： 一个字母的事，对端全错。

> **面试可以这么说**：我们有条 OTA 取消的应答，平台收到的时间戳永远是 0。查下来是代码里写成了 `timeStamp`（大写 S），而协议字段是 `timestamp`，结构体里两个字段都存在，所以编译能过， 但真正序列化出去的那个一直是空。同一个文件里五处都写错了，是复制粘贴的。 所以我现在遇到"字段有值但对面收到空"，第一件事就是对着协议文档核字段名的拼写。

5-3 日志级别 4「不输出日志」被封掉 —— 因为开了就起不来

commit：`aeced6d`（2026-03-05，zhangenwei）· `om_component.cc`

现象

运维平台下发"日志级别 = 4（不输出日志）"，设备重启后**运行异常 / 起不来。**

原来的写法

协议里日志级别是 5 档：

值	含义
0	DEBUG
1	INFO
2	WARN
3	ERROR
4	**NO_LOG（完全不输出）**

```
bool OM_COMPONENT::getWorkParamFromFile()
{
    switch (logLevelInt) {
        case 3: logLevel = LOG_LEVEL_ERROR; break;
        case 4: logLevel = LOG_LEVEL_NO_LOG; break;     // ← 支持"完全不输出"
        default: break;
    }
}
```

为什么错

`LOG_LEVEL_NO_LOG` 会让日志宏**连参数都不求值**（常见的优化写法是 `if (level > NO_LOG) stream << ...`）。于是：

- 某处代码把**有副作用的语句写在了日志宏的参数里**，日志关掉之后副作用消失 → 初始化少做一步 → 重启后起不来
- 或者某处 `if (log) {...}` 之外依赖了日志对象的生命周期

这是"日志宏吞掉副作用"的经典崩法。排查起来极难，因为**日志开着的时候一切正常**。

改法：把第 4 档映射成第 3 档

```
     case 4: //No-log
-        logLevel = LOG_LEVEL_NO_LOG;
+        logLevel = LOG_LEVEL_ERROR;
         break;
```

下发侧也一样，把内部状态 `LOG_LEVEL_NO_LOG` 上报成 "3" 而不是 "4"：

```
     case LOG_LEVEL_NO_LOG:
-        logLevelStr = "4";
+        logLevelStr = "3";
         break;
```

也就是说：协议里保留了第 4 档，但实现上不再真正关闭日志，最少保留 ERROR。

能学到什么

- **"允许完全关掉日志"是个危险选项。**它会让所有"日志宏里有副作用"的代码路径改变行为 —— 而这种代码在项目里几乎必然存在（比如 `LOG << initSomething()`）。
- 正确的做法有两步： ① 把日志宏的参数改成**纯表达式**（副作用单独一行）； ② 兜底做法就是这里 —— **保留最低档 ERROR，不提供真•全关。**
- 这类修法谈不上优雅（协议档位和实现对不上了），但**它是最低风险的选择**： 改一行映射，比翻遍全项目找出所有"日志里有副作用"的地方要现实。
- **值得记住的工程判断**：遇到"某个配置一改就崩"，先看这个配置会不会改变**代码路径**， 而不只是改变输出。

> **面试可以这么说**：我们的日志级别是协议下发的，有一档是"完全不输出"。现场发现下发这一档之后， 设备重启就起不来了。原因是有些日志宏的参数里带了有副作用的表达式，日志一关，副作用也没了， 初始化就少做了一步。我们最后的处理是把"完全不输出"这一档映射成只保留 ERROR， 等于不再真正关闭日志。这个修法不优雅，但比翻遍全项目清副作用现实。 这件事让我记住：有些配置改的不只是输出，是代码路径。

规律：这几个模块的错，集中在四个地方

规律一：改了一处，忘了配套的另外三处（占比最高）

这个项目里 5 个案例都是这个病：

改了什么	忘了什么	案例
结构体加字段	序列化 / 反序列化 / 打印	1-2
结构体加字段	构造函数里的默认值	1-3
批量赋值	变量名没改	1-4
topic 定义搬家	订阅列表没跟着改	2-2
协议字段改名	旧字段没删，新代码写到旧字段上	5-2

共同点：编译器兜不住。 这几处"配套修改"分散在不同文件、不同函数里， 只要有一处漏了，代码照样编、照样跑，只是行为不对。

防守办法：这类结构应该**用代码生成或者宏**消除"多处同步"这个动作。 这个项目里的 `JsonSerialize` 宏已经做了一半（序列化和反序列化成对）， 但打印那处还是手写的，所以 1-2 里就漏过。

规律二：连接类问题，一定要区分"物理连上"和"业务可用"

`3-1` 一条就踩了三个：

1. 轮询本地状态 ≠ 链路真实状态
2. TCP 连上 ≠ 订阅恢复（收不到指令）
3. clientId 相同 → broker 互相踢

共同点：所有"连接正常但业务不通"的问题，都能用这三条套一遍。 排查顺序建议：先看 clientId 唯一性（最便宜）→ 再看重连后有没有重订阅 → 最后才怀疑链路。

规律三：退出路径是最容易出事的地方

`4-3` / `4-4` / `4-5` 三条都在退出或启动阶段：

- 启动：线程起了但对象还没 new 出来（缺就绪屏障）
- 退出：`join` 等一个永远不会返回的 `wait`（谓词没含退出条件）
- 退出：跨线程析构有线程亲和性的对象（assert / 段错误）

共同点：这些代码在"正常运行"时一行都不会执行，所以测试覆盖不到。 而且它们出问题的表现是"重启卡住/升级失败"，现场最痛。

防守办法：启动加就绪屏障 + 超时；退出时给所有 `wait` 的谓词加 `|| stopping`； 有线程亲和性的对象要么在自己线程销毁、要么故意泄漏。这三条是固定套路，可以直接抄。

规律四：数量一致性没人检查

`3-3` 里 10 个 channel 拆了 9 个，剩下那对共用 —— **没有任何机制会发现这件事。**

同类问题还有：`4-4` 里的 `waitEventLoopsReady(7)` 是硬编码的 7， 以后加到 8 个 EventLoop，这个 7 不会自己变。

共同点：代码里存在"N 个这个东西"和"M 个那个东西"必须相等，但这个相等关系只存在人脑子里。

防守办法：加一句运行期断言（`assert(readers.size() == configs.size())`）， 或者在构造时用同一个列表驱动两边。比注释写"注意这里要同步"有效得多。

附一：本文引用的全部提交

commit	日期	作者	内容	对应案例
`994ac52`	2026-09-10	libo	修 ccindex 三处崩溃：非法 JSON double free、EventLoop 空指针、退出卡死	4-1 ~ 4-5
`0c0e685`	2026-08-27	root	mem-total/used/free 单位改为 B	5-1
`267fb52`	2026-05-09	zhangenwei	道路遗撒模块命名重复	1-1
`8c14386`	2026-04-29	hezhiyong	OTA 取消应答参数命名错误（timeStamp→timestamp）	5-2
`64e1682`	2026-04-15	zhangenwei	BUG25800 运维云连接错误、运维掉线	3-1
`6daadbe`	2026-04-03	zhangenwei	BUG25534 mec 自检相同 channel 数据丢失；自检阈值重标定	1-2 / 1-4 / 3-4
`4fea6d5`	2026-03-23	zhangenwei	mec 自检动静数据竞争，细化锁颗粒度	4-6
`f52f0ed`	2026-03-20	zhangenwei	mec 状态监控加锁	4-6
`1b0b4b5`	2026-03-16	zhangenwei	统一 channel 数据过大 → 拆成多个 channel	3-2 / 3-3
`0fc1d6b`	2026-03-11	zhangenwei	mec 自检初始化漏字段	1-3
`aeced6d`	2026-03-05	zhangenwei	日志级别 4 导致重启失败	5-3
`4767f67`	2026-03-05	zhangenwei	bug24325 相机内外参主题不对	2-2
`eb975a2`	2025-12-03	hezhiyong	BUG23650 大华相机违法图片只收 1 张	2-1
`de23348`	2025-07-08	changxuhui	相机接入模块优化（清理死配置项）	2-3

另外还有 10 个已抽取但本文未展开的（在 `fixes/` 目录里）：

commit	内容
`14bb558` / `5b9234f`	网络相机接入工厂类 + framework/module 实现（新增功能，非修错）
`e1abc59`	ccindex 中 mec 自检的 channel 修改
`1f55135`	MEC 自检动静数据监控逻辑重构
`599fa9a`	雷达点云监控拆分成"数据监控"和"链路监控"
`74d4205`	bug24321 MEC 自检不符合接口
`9672c84`	bug24323/24324 运维协议
`f1fc634`	bug24320 OTA 不能升级
`d4efa1c`	相机和雷达升级取消逻辑
`f52f0ed`	状态监控加锁（已并入 4-6）

附二：和你简历三条的对应关系

> ⚠️ **这一节是最重要的。**这些提交**都不是你做的**（你在这个仓库里没有提交记录）， 所以面试时只能说"我读代码/看提交历史时了解到"，**不能说"我修的"。**

简历原文	本文对应案例	面试可以说到什么程度
设备抽象与插件化接入	1-1 ~ 1-4	可以说"插件层代码是各人各加一段，合并残留导致编译错误很常见"；**不能说"我设计了这个框架"**
多路视频流接入	2-1 ~ 2-3	可以说"我们接过 RTSP 相机，XML 遍历的循环变量写错过，导致只处理第一张图"；这是 hezhiyong 修的，说"团队踩过"
设备与平台运维（MQTT 上报）	3-1 ~ 3-4	可以说"连接状态判断踩过轮询的坑、重连后要重订阅、clientId 要带设备号"；这三条**你在自己做设备接入时大概率也遇到过**，可以讲成经验
（通用）常驻进程退出	4-1 ~ 4-5	可以在被问"你处理过什么棘手问题"时讲"条件变量退出卡死"（这条和 traffic_flow 是同型，你自己的代码里也踩过）
（通用）协议字段	5-1 ~ 5-3	讲"字段名大小写/单位标签"这类，属于通用经验，安全

最安全的讲法：把它们变成**"我知道的几个坑"，而不是"我做的事"。** 面试官问"你怎么保证可靠性"，你可以答"我们项目里出过这几类问题，所以我会在 xx 地方加 xx 检查" —— 这是**用踩坑历史支撑工程判断**，比吹自己做了多少功能可信得多。

怎么继续挖

```
# 1. 看某个案例的完整 diff
cd D:/ZhiLuOS/60airos2.0-git历史/fixes
ls                                    # 24 个 md，每个含完整 diff

# 2. 上 193 机查更多
ssh 172.20.65.193
cd /work/sunboxue/airos2.0
git show 64e1682                                                  # 某次提交全文
git log --oneline -- middleware/protocol/om/om_component.cc     # 某文件全部历史
git log -S "maxTimeoutTolerance" --oneline --all                  # 某个字段什么时候加的

# 3. 按模块找 fix（本地就能做）
cd D:/ZhiLuOS/60airos2.0-git历史
grep -i "device_service" 03-提交文件变更.tsv | grep -i fix | head -30
grep -i "ipcamera" 03-提交文件变更.tsv | awk -F'\t' '{print $4}' | sort -u
```

`git log -S`（pickaxe）是挖"这个字段什么时候加的、为什么加"最好的工具。 比如查 `maxTimeoutTolerance` 是怎么一步步引入的：

```
git log -S "maxTimeoutTolerance" --oneline --all
git log -G "maxTimeoutTolerance" -p --oneline | head -200
```